

Ejercicios UT7 –Utilización avanzada de clases.

Programas en los que aparecen la composición, la agregación y/o la asociación.

1. La competencia en materia de impuestos sobre vehículos de tracción mecánica pertenece en la actualidad a los ayuntamientos. Con objeto de gestionar el cobro de dicho impuesto, el ayuntamiento de “Elizabethtown” (Kentucky) dispone de la información de las personas empadronadas en él (id, nombre, apellidos, dirección, ciudad, provincia, teléfono y Email) y de los vehículos sujetos al impuesto (id, año, modelo, motor y combustible). Desarrolle las clases necesarias para gestionar el cobro durante ejercicio tributario actual utilizando un modelo relacional y su alternativa utilizando un modelo orientado a objetos.
2. Modifique el diseño anterior incorporando la posibilidad de gestionar el cobro durante cualquier ejercicio tributario.

Programas en los que aparece la herencia simple

3. Programe las clases Persona y Trabajador de acuerdo con las especificaciones:

Persona

- Atributos privados (nombre, teléfono y edad).
- Constructor sin argumentos.
- Constructor con 3 parámetros para las 3 variables de instancia.
- Setters y getters.
- Método toString.

Trabajador (subclase de persona):

- Atributos: categoría profesional (A, B ó C), Antigüedad (int).
- Constructores: sin y con parámetros, incluso los de la clase Persona.
- Métodos setters y getters.
- Método toString.

Desarrolle una clase Main que contendrá el método main, en la que se creen dos objetos de la clase Trabajador. El primero se instanciará con el constructor sin argumentos y posteriormente se introducirá el contenido a sus atributos utilizando los correspondientes métodos setters. El segundo se instanciará utilizando el constructor que utiliza todos los parámetros y al que se le pasará como valores la información leída desde teclado. A estos objetos de la clase Trabajador, también se les dará valores a sus atributos nombre, teléfono y edad que heredan de la clase Persona.

Visualice su contenido utilizando el método toString() que ha programado y visualice el nombre del trabajador de más antigüedad.

4. Crear un programa para gestionar el inventario de una tienda de mascotas. De cada mascota se deberá guardar nombre y fecha de nacimiento. Habrá 3 tipos de mascotas: perros, gatos y loros. La configuración de las clases será la siguiente:

- De los perros se almacenan los atributos raza y pulgas(true/false). Y deberá tener un método que sea emiteSonido que indicará el sonido que hace el perro. Además, será posible mostrar todas sus características.
- De los gatos se almacenarán los atributos peloLargo(true/false) y color. Y deberá tener un método que sea emiteSonido. Además, será posible mostrar todas sus características.
- De los loros se necesitan los atributos origen y habla(true/false). Y deberá tener un método que sea emiteSonido, otro que sea volar y otro saluda. Además, será posible mostrar todas sus características.
- Crear una clase principal en la que se creen distintas instancias de las mascotas y se llamen a sus distintos métodos.

Programas en los que aparece la herencia simple y el polimorfismo.

5. Desarrolle una aplicación que permita la Gestión de las nóminas de una empresa.
En la empresa existen varios tipos de Trabajadores. De todos ellos nos interesa el dni, nombre, salario base y el salario final, que se calcula en función del tipo de trabajador y de los complementos. De manera que salario final es el salario base más un complemento.

Los trabajadores se dividen en dos grupos: informáticos y gestión. De los informáticos nos interesa aparte de los datos citados, la titulación. Hay dos tipos de informáticos: Analistas: el complemento es del 30% y Programadores: el complemento es del 15%. Del personal de gestión nos interesa la antigüedad en la empresa. Hay dos tipos de personal de gestión: Administrativos: el complemento es fijo y supone 20€ por cada año de antigüedad y Auxiliares: el complemento es fijo de 100€ independiente de la antigüedad. Recuerde que el salario final es el salario base más un complemento. Por lo tanto, desarrolle:

- La clase Trabajador con los atributos vistos anteriormente y sus clases derivadas Informático, Gestión, Analista, Programador, Administrativo y Auxiliar.
- La clase Empresa con CIF y un nombre. También tendrá un array de Trabajadores. Y el método necesario para añadir los trabajadores a la empresa.
- La clase Aplicación que hará lo siguiente:
 - Dará de alta una empresa (no hace falta pedir datos al usuario, crearlo directamente)
 - Añadir 2 trabajadores de cada tipo e introducirlos en la empresa que se ha creado previamente (no hace falta pedir datos al usuario, crearlo directamente).
 - Mostrar un listado de Trabajadores junto con su salarioBruto y salarioFinal.

En el listado deben aparecer los trabajadores junto con su tipo al inicio:

Analista: {dni=204433963W, nombre=Carles, salarioB=2800.0, salarioF=3640.0}
 Analista: {dni=204437956R, nombre=Maria, salarioB=3000.0, salarioF=3900.0}
 Programador: {dni=204437963W, nombre=Alba, salarioB=1100.0, salarioF=1265.0}
 Programador: {dni=204437964W, nombre=Marti, salarioB=1200.0, salarioF=1380.0}
 Administrativo: {dni=20443300A, nombre=Nuria, salarioB=1300.0, salarioF=1400.0}
 Administrativo: {dni=20443301A, nombre=Pepe, salarioB=1400.0, salarioF=1600.0}

Auxiliar: {dni=20443302A, nombre=Jose, salarioB=1000.0, salarioF=1100.0}
Auxiliar: {dni=20443303A, nombre=Ana, salarioB=1100.0, salarioF=1200.0}

Programas sin especificar el o los tipos de relaciones

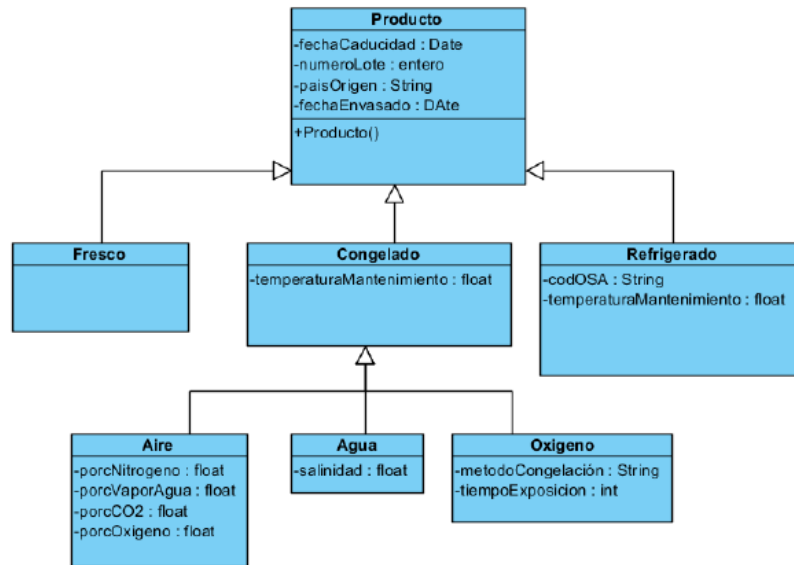
6. Los vehículos de la policía nacional denominados coches patrulla o vehículos tipo Z tienen en la parte superior de su carrocería un código formado por tres caracteres (letras y dígitos) que permite, por su ubicación y dimensiones, una fácil identificación desde localizaciones elevadas. En cada turno, el vehículo es asignado a dos funcionarios de la brigada de seguridad ciudadana identificados cada uno de ellos por su número de placa. En la actualidad existen tres turnos (00:00 a 07:59, 8:00 a 15:59, 16:00-23:59). Desarrolle un modelo de datos y a partir de él, un programa que permita conocer quienes ocupaban un vehículo a partir de una fecha y hora concretas. No es necesario que programe las operaciones CRUD de las clases que identifique.
7. Se plantea desarrollar un programa Java que permita la gestión de una empresa agroalimentaria que trabaja con tres tipos de productos: productos frescos, productos refrigerados y productos congelados. Todos los productos llevan esta información común: fecha de caducidad y número de lote. A su vez, cada tipo de producto lleva alguna información específica. Los productos frescos deben llevar la fecha de envasado y el país de origen. Los productos refrigerados deben llevar el código del organismo de supervisión alimentaria, la fecha de envasado, la temperatura de mantenimiento recomendada y el país de origen. Los productos congelados deben llevar la fecha de envasado, el país de origen y la temperatura de mantenimiento recomendada

Hay tres tipos de productos congelados: congelados por aire, congelados por agua, congelados por nitrógeno. Los productos congelados por aire deben llevar la información de la composición del aire con que fue congelado (% de nitrógeno, % de oxígeno, % de dióxido de carbono y % de vapor de agua). Los productos congelados por agua deben llevar la información de la salinidad del agua con que se realizó la congelación en gramos de sal por litro de agua. Los productos congelados por nitrógeno deben llevar la información del método de congelación empleado y del tiempo de exposición al nitrógeno expresada en segundos.

Crear el código de las clases Java implementando una relación de herencia siguiendo estas indicaciones:

- a) En primer lugar, realizar un esquema con papel y bolígrafo (o una herramienta tipo VisualParadigm o similar) donde se represente cómo se van a organizar las clases cuando escriba el código. Estudiar los atributos de las clases y trasladar a la superclase todo atributo que pueda ser trasladado.
- b) Crear superclases intermedias (aunque no se correspondan con la descripción dada de la empresa) para agrupar atributos y métodos cuando sea posible. Esto corresponde a “realizar abstracciones” en el ámbito de la programación, que pueden o no corresponderse con el mundo real.
- c) Cada clase debe disponer de constructor y permitir establecer (set) y recuperar (get) el valor de sus atributos y tener un método que permita mostrar la información del objeto cuando sea procedente.

Pista:



Crear una clase testHerencia con el método main donde se creen: 2 productos frescos, 3 productos refrigerados, 5 productos congelados (2 de ellos congelados por agua, otros 2 por aire y 1 por nitrógeno). Mostrar la información de cada producto por pantalla.

Programas que incorporan clases abstractas.

8. En este ejercicio se trata de crear una aplicación de gestión de personal de una empresa. Del personal se conocen: DNI, Nombre, Apellidos y Año de alta en la empresa. Hay dos tipos de empleados:
 - Empleados con salario fijo: estos empleados tienen un sueldo base y un porcentaje adicional en función del número de años que lleven en la empresa con los siguientes cálculos: Menos de 2 años solo tienen salario base. Entre 2 y 3 años un 7% del salario base. Entre 4 y 8 años un 11% del salario base y entre 9 y 15 años un 17% del salario base.
 - Empleados a comisión: estos empleados tendrán todos un salario fijo de 950 €, un número de clientes captados y una comisión por cada cliente captado. El sueldo es el resultado de multiplicar el número de clientes captados por la comisión por cliente. Si el resultado de este cálculo no llega al salario fijo, se cobra solo el salario fijo.

Requisitos de la aplicación:

- Clase padre Empleado que no se podrá instanciar.
 - Clases EmpleadoAsalariado y EmpleadoComision que heredarán de Empleado.
- Todas las clases deberán tener un constructor con parámetros y un constructor por defecto. La clase Empleado tendrá un método imprimir() y un método abstracto getSalario(). Clase principal donde se creará un array en el método main con los siguientes datos:

- Antonio López. DNI: 6546546Z, año de alta: 2014, salario fijo: 1125
- Sandra Nieto. DNI: 7879879S, año de alta: 2011, 169 clientes captados a 7.10 cada uno.
- Manuel Ruíz. DNI: 4654654D, año de alta: 2010, 35 clientes captados a 6.90 cada uno.
- María Ramos. DNI: 77879878F, año de alta: 2011, salario fijo: 1055

Los dos primeros empleados se crearán utilizando el constructor con parámetros. Los dos últimos con el constructor por defecto y los getters y setters correspondientes.

Desde el main de la clase principal se llamarán a dos métodos: `sueldoMayor()`. Este método recibirá un array de tipo `Empleado` por parámetros y devolverá el nombre apellido y salario del Empleado con el salario más alto. `mostrarTodos()`. Este método recibirá por parámetro un array de tipo `Empleado` y mostrará los datos de todos los Empleados del array.